JAVASCRIPT · 1.ª EVALUACIÓN · EJERCICIOS TIPO EXAMEN

5 ejercicios integradores de Entorno Cliente

Cinco enunciados completos para demostrar que sabes unir lo trabajado: datos, funciones, objetos, eventos, formularios y DOM en aplicaciones que funcionan en el navegador.

Temas 1–4 · Fundamentos, arrays y funciones | Tema 5 · POO | Tema 6 · Eventos y formularios | sin soluciones

↗ Condiciones de trabajo

1. **Cada ejercicio es una prueba independiente** valorada sobre 10 puntos. Crea una carpeta por ejercicio (`EX1`, `EX2`…) con su `index.html` y su `app.js`.
2. **Parte del HTML dado.** Puedes añadir clases o CSS, pero conserva los `id` y los `name`. Escribe tu lógica en `app.js`.
3. **Usa JavaScript sin librerías.** Los datos viven en memoria: al recargar, la aplicación vuelve al estado inicial.
4. **Prueba antes de entregar.** Completa cada tabla de casos de prueba y no dejes errores en la consola. Debes poder explicar y modificar tu código en la defensa final.

Recorrido de los ejercicios

Ejercicio	Tiempo	Qué integra
EX1 · Clasificación de una carrera	60 min	Funciones, arrays de objetos, filtros y renderizado.
EX2 · Préstamos de la biblioteca	75 min	Herencia, encapsulación, polimorfismo y delegación.
EX3 · Citas de un taller mecánico	75 min	Validación nativa, regex, reglas propias y FormData.
EX4 · Tablón de incidencias	75 min	Traza de eventos, depuración y ampliación de código ajeno.
EX5 · Taquilla de un festival	100 min	Aplicación completa: modelo, alta, edición, filtros y aforo.

Los tiempos son orientativos. Dificultad progresiva: EX1 es el más guiado y EX5 el más abierto.

00 Qué evalúa cada ejercicio

Contenido	EX1	EX2	EX3	EX4	EX5
Tipos, conversiones, decisiones y bucles	•	•	•	•	•
Arrays de objetos y funciones con `return`	•	•	•	•	•
`filter`, `map`, `find` y `reduce`	•	•	•	•	•
Clases, `extends`, `super` y polimorfismo		•	•		•
Campos `#`, `get`, `set` y `static`		•	•	•	•
Recorridos, `instanceof` y prototipos		•			•
Propagación, `target` y `currentTarget`				•	
`input`, `change` y teclado	•	•	•	•	•
`submit`, `FormData` y validación nativa			•	•	•
Expresiones regulares y `setCustomValidity`		•	•	•	•
Delegación y modelo → renderizado	•	•	•	•	•
Depuración con DevTools				•	•

Criterios comunes de corrección

Criterio	Qué se observa
Funcionamiento	Resuelve los casos normales, los límites y las entradas inválidas sin errores en consola.
Modelo y lógica	Los datos y las reglas están en funciones o clases; no toda la lógica vive en los manejadores.
DOM y eventos	La interfaz se pinta desde el modelo, con listeners adecuados y sin duplicarlos.
Calidad	Nombres claros, `const` / `let` correctos, funciones pequeñas y texto de usuario mostrado con `textContent`.
Comprensión	Explica el flujo de una interacción y responde a las preguntas de defensa.

Qué pesa más

Un detalle menor de sintaxis o de aspecto no cuenta igual que no comprender el flujo de eventos o dejar el modelo y el DOM en estados distintos. Cada parte puntúa lo que funcione y puedas justificar.

EX1 Clasificación de una carrera popular

10 puntos | 60 minutos | Dificultad: media

Situación: la organización de una carrera de 10 km quiere consultar los resultados en una página. Los datos ya están ordenados por tiempo de llegada, y las personas retiradas aparecen al final con `segundos: null`.

INTEGRA · TEMAS 1–4 · TEMA 6 §07–§09 · §17–§18

HTML Material de partida

Punto de partida: copia este cuerpo en un documento con `<script src="app.js" defer>` y estos datos al inicio de `app.js`.

HTML · cuerpo de index.html

```html
<h1>Carrera popular · 10 km</h1>
<label for="categoria">Categoría</label>
<select id="categoria">
  <option value="todas">Todas</option>
  <option value="sub23">Sub-23</option>
  <option value="senior">Sénior</option>
  <option value="veterano">Veterano</option>
</select>
<label for="buscar">Buscar por nombre</label>
<input id="buscar" type="search">
<label><input id="retirados" type="checkbox"> Mostrar retirados</label>
<p id="resumen" role="status"></p>
<p id="vacio" hidden>Ningún resultado coincide con los filtros.</p>
<table>
  <thead>
    <tr><th>Pos.</th><th>Dorsal</th><th>Nombre</th><th>Categoría</th>
        <th>Tiempo</th><th>Ritmo</th><th>Ficha</th></tr>
  </thead>
  <tbody id="tabla"></tbody>
</table>
<p id="ficha"></p>
```

JavaScript · datos iniciales de app.js

```javascript
const DISTANCIA_KM = 10;
const corredores = [
  { dorsal: 101, nombre: "Mikel Arana", edad: 28, segundos: 2462 },
  { dorsal: 102, nombre: "Ane Etxeberria", edad: 34, segundos: 2530 },
  { dorsal: 103, nombre: "Lucía Romero", edad: 19, segundos: 2605 },
  { dorsal: 104, nombre: "Jon Ugalde", edad: 45, segundos: 2688 },
  { dorsal: 105, nombre: "Nerea Goñi", edad: 22, segundos: 2755 },
  { dorsal: 106, nombre: "Iker Soto", edad: 41, segundos: 2911 },
  { dorsal: 107, nombre: "Maite Olano", edad: 52, segundos: 3067 },
  { dorsal: 108, nombre: "Unai Prieto", edad: 17, segundos: 3120 },
  { dorsal: 109, nombre: "Leire Vidal", edad: 30, segundos: null },
  { dorsal: 110, nombre: "Asier Mora", edad: 48, segundos: null }
];
```

EX1 Se pide

PARTE A · LÓGICA SIN DOM (4 PUNTOS)

1. `obtenerCategoria(edad)` devuelve `"sub23"` por debajo de 23 años, `"senior"` de 23 a 39 y `"veterano"` a partir de 40. **0,5 p**
2. `formatearTiempo(segundos)` devuelve `"mm:ss"` con dos cifras en los segundos. Si recibe `null`, devuelve `"Retirado"`. **1 p**
3. `calcularRitmo(segundos)` devuelve el ritmo por kilómetro redondeado al segundo, con el formato anterior, o `"—"` si no terminó. **0,5 p**
4. `asignarPosiciones(lista)` devuelve una lista nueva con una propiedad `posicion` en quienes han terminado y `null` en quienes se han retirado. No modifica los objetos originales. **1 p**
5. `calcularResumen(lista)` devuelve un objeto con el número de personas de la lista, las que han terminado, su media de segundos y el objeto con el mejor tiempo. Con ninguna persona clasificada, la media será `null` y el mejor tiempo también. **1 p**

PARTE B · RENDERIZADO (2 PUNTOS)

1. `renderizar()` reconstruye las filas de `#tabla` desde los datos, usando métodos del DOM y `textContent`. Cada fila incluye un botón «Ver ficha» con su dorsal en `data-dorsal`. **1,5 p**
2. Muestra en `#resumen` el texto `Visibles: X · Media: mm:ss · Mejor: Nombre (mm:ss)`, calculado sobre las filas visibles. La media se redondea al segundo. Sin tiempos válidos, escribe `Sin tiempos` en lugar de la media y el mejor tiempo. **0,5 p**

PARTE C · FILTROS COMBINADOS (3 PUNTOS)

1. Elige el evento adecuado para cada control. Los tres filtros actúan juntos y no alteran el array original. **1,5 p**
2. La búsqueda ignora mayúsculas y espacios exteriores. La posición de cada persona es siempre la de la clasificación general. **0,5 p**
3. `Escape` en el campo de búsqueda lo vacía y actualiza la tabla. Asignar `value` por código no dispara `input`. **0,5 p**
4. `#vacio` solo se muestra cuando no hay resultados. **0,5 p**

PARTE D · DELEGACIÓN (1 PUNTO)

1. Con una sola escucha en `#tabla`, «Ver ficha» escribe en `#ficha`: `Nombre · dorsal X · N años · categoría`. Debe funcionar tras filtrar y al pulsar un elemento interior del botón. **1 p**

OK Casos de prueba

Acción o llamada	Resultado esperado
`obtenerCategoria(22)` · `(23)` · `(39)` · `(40)`	`sub23` · `senior` · `senior` · `veterano`
`formatearTiempo(2462)` · `(605)` · `(null)`	`41:02` · `10:05` · `Retirado`
`calcularRitmo(2462)`	`04:06`
Estado inicial	`Visibles: 8 · Media: 46:07 · Mejor: Mikel Arana (41:02)`
Categoría Veterano	3 filas con posiciones 4, 6 y 7. Media `48:09`; mejor, Jon Ugalde.
Veterano + Mostrar retirados	4 filas; Asier Mora aparece sin posición y no cambia la media.
Buscar `" ANE "`	Solo Ane Etxeberria, en posición 2.
Sub-23 + buscar `asier` + Mostrar retirados	Ninguna fila, aparece `#vacio` y el resumen indica `Sin tiempos`.
Retirados activos + buscar `leire`	Una fila, resumen con `Sin tiempos` y ficha disponible.

Defensa: ¿Por qué calculas las posiciones antes de filtrar? ¿Qué falla si escuchas `keydown` en lugar de `input` para buscar? ¿Qué demuestra que no has modificado `corredores`?

EX2 Préstamos de la biblioteca

10 puntos | 75 minutos | Dificultad: media-alta

Situación: la biblioteca del centro presta libros, revistas y videojuegos. Cada tipo tiene un plazo y una multa diferentes, y cada título puede tener varios ejemplares. Hay que modelarlo con clases y conectarlo a una interfaz.

INTEGRA · TEMA 5 · TEMAS 3–4 · TEMA 6 §15 · §17–§18

HTML Material de partida

HTML · cuerpo de index.html

```html
<h1>Biblioteca del centro</h1>
<label for="tipo">Tipo</label>
<select id="tipo">
  <option value="todos">Todos</option>
  <option value="libro">Libros</option>
  <option value="revista">Revistas</option>
  <option value="videojuego">Videojuegos</option>
</select>
<label for="diasUso">Días que ha durado el préstamo</label>
<input id="diasUso" type="number" min="1" value="1">
<p id="estadisticas"></p>
<p id="mensaje" role="status"></p>
<p>Multas cobradas: <strong id="caja">0.00 €</strong></p>
<ul id="catalogo"></ul>
```

REGLAS DE CADA TIPO

Clase	Datos propios	Días máximos	Multa por día de retraso
`Libro`	`autor`, `paginas`	21	0,20 €
`Revista`	`numero`	7	0,10 €
`Videojuego`	`plataforma`, `pegi`	5	0,50 €

JavaScript · datos para crear el catálogo

```javascript
const datos = [
  new Libro("LIB-001", "Patria", 2, "Fernando Aramburu", 648),
  new Libro("LIB-002", "Obabakoak", 1, "Bernardo Atxaga", 432),
  new Revista("REV-014", "Muy Interesante", 3, 512),
  new Revista("rev-99", "Código incorrecto", 1, 7),
  new Videojuego("JUE-007", "Portal 2", 1, "PC", 12)
];
```

EX2 Se pide

PARTE A · CLASE BASE (3 PUNTOS)

1. `Material` recibe `codigo`, `titulo` y `ejemplares`. Guarda los disponibles en un campo privado `#disponibles` que empieza igual que `ejemplares`, y ofrece un `get disponibles` sin `set`. **1 p**
2. El título usa `get` y `set`: un valor con menos de 2 caracteres tras `trim` se ignora y conserva el anterior. **0,5 p**
3. El método estático `Material.codigoValido(codigo)` comprueba tres letras mayúsculas, un guion y tres dígitos. **0,5 p**
4. `prestar()` resta un disponible y devuelve `true`; sin disponibles, devuelve `false`. `devolver(dias)` suma uno y devuelve la multa; si no había ejemplares prestados, devuelve `null` sin cambiar nada. **1 p**

PARTE B · HERENCIA Y POLIMORFISMO (3 PUNTOS)

1. Crea `Libro`, `Revista` y `Videojuego` con `extends` y `super`. Cada una sobrescribe `diasMaximos()`, `multaDiaria()` y `tipo()`. **1,5 p**
2. `calcularMulta(dias)` se escribe una sola vez en `Material` y multiplica los días de retraso por la multa diaria. **0,5 p**
3. `resumen()` de la base devuelve `codigo · titulo · disponibles/ejemplares`. Cada hija lo amplía usando `super.resumen()`, sin repetir ese texto. **1 p**

PARTE C · INTERFAZ (3 PUNTOS)

1. Forma `catalogo` solo con los materiales de código válido y renderízalo en `#catalogo` mediante `textContent`. Cada fila tiene «Prestar» y «Devolver» con `data-accion` y `data-codigo`. **1 p**
2. Usa una única escucha delegada. Tras cada acción, actualiza `#mensaje`, la caja acumulada con dos decimales y la lista. `#diasUso` vacío o menor que 1 no devuelve nada y muestra un aviso. **1 p**
3. El filtro `#tipo` se apoya en `instanceof`. `#estadisticas` muestra siempre el catálogo completo: `Libros: X · Revistas: Y · Videojuegos: Z · Disponibles: A de B`. **1 p**

PARTE D · COMPROBACIÓN POR CONSOLA (1 PUNTO)

1. Añade `Material.prototype.centro = "Arangoya"`. Recorre un libro con `for...in` y con `Object.getOwnPropertyNames()`. Anota qué diferencia aparece y por qué no sale `#disponibles`. **0,5 p**
2. Añade a una instancia una propiedad `oferta`, elimínala con `delete` y comprueba que las demás siguen intactas. **0,5 p**

OK Casos de prueba

Acción o llamada	Resultado esperado
Estado inicial	4 filas. `Libros: 2 · Revistas: 1 · Videojuegos: 1 · Disponibles: 7 de 7`
Libro: `calcularMulta(21)` · `(25)`	`0` · `0.8`
Revista: `calcularMulta(7)` · `(10)`	`0` · `0.3` (dos decimales en pantalla)
Videojuego: `calcularMulta(8)`	`1.5`
Prestar Portal 2 dos veces	La segunda avisa de que no quedan ejemplares y disponibles sigue en 0.
Devolver Patria sin haberlo prestado	Mensaje de aviso; la caja no cambia.
Prestar y devolver Portal 2 con 8 días, y Patria con 25	Caja: `2.30 €`. Disponibles: 7 de 7.
Filtrar Revistas y prestar «Muy Interesante»	Se ve solo esa fila con 2/3; las estadísticas indican `6 de 7`.
`libro.titulo = " "` · `libro.disponibles = 99`	El título y los disponibles no cambian.

Defensa: ¿Dónde está guardado `calcularMulta` y cómo lo encuentra un `Videojuego`? ¿Por qué `this.diasMaximos()` ejecuta una versión distinta en cada objeto? ¿Por qué contar con `instanceof Material` no serviría para separar tipos?

EX3 Citas de un taller mecánico

10 puntos | 75 minutos | Dificultad: media-alta

Situación: un taller quiere recoger citas desde su web. El formulario debe impedir datos incorrectos, calcular el precio mientras se rellena y mantener una lista de citas que se pueda anular.

INTEGRA · TEMA 6 §09 · §11–§18 · TEMA 5 · TEMAS 1–4

HTML Material de partida

HTML · cuerpo de index.html

```html
<h1>Pide cita en el taller</h1>
<form id="formCita" novalidate>
  <label for="matricula">Matrícula (1234BCD)</label>
  <input id="matricula" name="matricula" required>
  <label for="correo">Correo electrónico</label>
  <input id="correo" name="correo" type="email" required>
  <label for="correo2">Repite el correo</label>
  <input id="correo2" name="correo2" type="email" required>
  <label for="km">Kilómetros del vehículo</label>
  <input id="km" name="km" type="number" required>
  <label for="servicio">Servicio</label>
  <select id="servicio" name="servicio" required>
    <option value="">Elige un servicio</option>
    <option value="aceite">Cambio de aceite (60 €)</option>
    <option value="revision">Revisión completa (90 €)</option>
    <option value="frenos">Frenos (120 €)</option>
  </select>
  <fieldset>
    <legend>Extras</legend>
    <label><input type="checkbox" name="extras" value="lavado"> Lavado (15 €)</label>
    <label><input type="checkbox" name="extras" value="sustitucion"> Coche de sustitución (30 €)</label>
  </fieldset>
  <label><input id="domicilio" type="checkbox" name="domicilio"> Recogida a domicilio (12 €)</label>
  <label for="direccion">Dirección de recogida</label>
  <input id="direccion" name="direccion" disabled>
  <p id="previa" role="status">Precio estimado: —</p>
  <button type="submit">Reservar cita</button>
  <button type="reset">Vaciar</button>
</form>
<p id="mensaje" role="status"></p>
<p id="resumen"></p>
<ul id="citas"></ul>
```

Tu trabajo en el HTML

Añade los atributos de validación que necesites en `matricula` y `km`. El resto de reglas se resuelve desde JavaScript.

EX3 Se pide

PARTE A · MODELO (2,5 PUNTOS)

1. Crea la clase `Cita` con `id`, `matricula`, `correo`, `km`, `servicio`, `extras` (array) y `direccion`. **0,5 p**
2. Crea `Cita.precioServicio(servicio)` como método estático con `switch`. **0,5 p**
3. El getter `recargoKm` es verdadero desde 150 000 km. `precioTotal()` suma servicio y extras, aplica un 10 % de recargo si procede y añade 12 € si hay dirección. La recogida no lleva recargo. **1,5 p**

PARTE B · VALIDACIÓN (3,5 PUNTOS)

1. **Matrícula:** cuatro dígitos seguidos de tres consonantes mayúsculas, sin vocales ni `Ñ` ni `Q`. Exprésalo en `pattern`, sin barras delimitadoras. **1 p**
2. **Kilómetros:** entero entre 0 y 999 999, con atributos HTML. **0,5 p**
3. **Correos:** ambos deben coincidir ignorando mayúsculas y espacios exteriores. El error aparece en `correo2`. **0,5 p**
4. **Matrícula repetida:** no se admiten dos citas para una misma matrícula. **0,5 p**
5. **Dirección:** si se marca la recogida, se habilita y pasa a ser obligatoria, con al menos 8 caracteres tras `trim`. Si se desmarca, se deshabilita y se vacía. **0,5 p**
6. Usa `setCustomValidity` y `reportValidity`. Al editar, se retiran los errores propios; un dato ya corregido debe poder enviarse. **0,5 p**

PARTE C · FLUJO DEL FORMULARIO (2,5 PUNTOS)

1. Con una única escucha de `input` en el formulario, actualiza `#previa`. Mientras falten servicio o kilómetros, muestra `–`. Un `km` vacío no se interpreta como 0. **1 p**
2. Recoge el envío con `FormData` y `getAll`. Convierte los tipos, normaliza el correo a minúsculas y crea la cita. Un envío inválido no modifica el modelo. **1 p**
3. Después de un alta, y también con «Vaciar», el formulario queda como al cargar: precio en `–`, dirección deshabilitada y sin errores. **0,5 p**

PARTE D · LISTA DE CITAS (1,5 PUNTOS)

1. Renderiza `#citas` desde el array, con matrícula, servicio, extras y precio con dos decimales. En `#resumen`, muestra `Citas: X · Facturación: Y €` usando `reduce`. **1 p**
2. «Anular» funciona con delegación y `data-id`. La matrícula anulada vuelve a poder reservarse. **0,5 p**

OK Casos de prueba

Datos	Resultado esperado
Matrículas `1234BCD`, `0042XYZ`	Válidas.
`1234ABC`, `1234bcd`, `123BCD`, `12345BCD`, `1234BCÑ`, `1234BCQ`	Inválidas.
Aceite + lavado · 90 000 km	`75.00 €`
Revisión · 150 000 km · 149 999 km	`99.00 €` · `90.00 €`
Frenos + lavado + sustitución · 180 000 km	`181.50 €`
Frenos · 200 000 km · recogida	`144.00 €`
Servicio elegido y kilómetros `0`	Calcula el precio: 0 es un dato válido.
`Ana@Correo.com` y `ana@correo.com`	Coinciden. Se guarda `ana@correo.com`.
Dos citas con `1234BCD`	La segunda se rechaza. Tras anular la primera, se acepta.
Recogida marcada, dirección vacía; luego se desmarca	Primero inválido; después el envío funciona, sin dirección ni recargo de 12 €.
Altas de 75 €, 181,50 € y 144 €	`Citas: 3 · Facturación: 400.50 €`

Defensa: ¿Por qué `direccion` no aparece en `FormData` cuando está deshabilitada? ¿Por qué hace falta limpiar con `setCustomValidity("")`? ¿Qué regla no puede expresarse solo con atributos HTML y por qué?

EX4 Tablón de incidencias: traza, depuración y ampliación

10 puntos | 75 minutos | Dificultad: alta

Situación: heredas una aplicación que alguien dejó a medias. Primero demostrarás que entiendes el recorrido de un evento; después corregirás sus fallos con DevTools y, por último, la ampliarás.

INTEGRA · TEMA 6 §03–§06 · §12–§19 · TEMA 5 · TEMAS 1–4

A Parte A · Predice sin ejecutar (2 puntos)

Condición: responde por escrito antes de abrir el navegador. Después ejecuta el código y justifica cualquier diferencia.

HTML · estructura del laboratorio

```html
<section id="panel">
  <article id="tarjeta">
    <button id="accion" type="button"><span id="texto">Cerrar</span></button>
  </article>
</section>
```

JavaScript · todas las escuchas se registran en este orden

```js
const panel = document.querySelector("#panel");
const tarjeta = document.querySelector("#tarjeta");
const accion = document.querySelector("#accion");

panel.addEventListener("click", () => console.log("A"), { capture: true });

tarjeta.addEventListener("click", (e) => {
  console.log("B", e.target.id, e.currentTarget.id);
});

accion.addEventListener("click", (e) => {
  console.log("C");
  e.stopPropagation();
  console.log("D");
});

accion.addEventListener("click", () => console.log("E"));

panel.addEventListener("click", () => console.log("F"));

accion.addEventListener("click", () => console.log("G"), { once: true });
```

RESPONDE

1. Salida del primer clic sobre el texto «Cerrar». **0,5 p**
2. Salida de un segundo clic en el mismo sitio. **0,25 p**
3. Recarga y pulsa una zona de `tarjeta` fuera del botón. Escribe la salida con los valores de `e.target.id` y `e.currentTarget.id`. **0,5 p**
4. Recarga, cambia `stopPropagation` por `stopImmediatePropagation` y pulsa el texto. ¿Qué mensajes desaparecen y por qué? **0,5 p**
5. Si se pulsara el `span` y la escucha de `tarjeta` llegara a ejecutarse, ¿qué valdría `e.target.id`? ¿Y `e.currentTarget.id`? **0,25 p**

B Parte B · Depura la aplicación (5 puntos)

Punto de partida: copia estos dos archivos. Contienen 10 fallos, en el HTML o en JavaScript. Cada uno vale 0,5 puntos: 0,25 por corregirlo y 0,25 por explicarlo.

Código para depurar · index.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Tablón de incidencias</title>
  <script src="app.js"></script>
</head>
<body>
  <h1>Tablón de incidencias</h1>
  <form id="formIncidencia" novalidate>
    <label for="titulo">Título</label>
    <input id="titulo" name="titulo" required>
    <label for="prioridad">Prioridad (1 a 5)</label>
    <input id="prioridad" name="prioridad" type="number" min="1" max="5">
    <button type="submit">Añadir</button>
    <button id="limpiar">Limpiar</button>
  </form>
  <label for="filtro">Ver</label>
  <select id="filtro">
    <option value="todas">Todas</option>
    <option value="abierta">Abiertas</option>
    <option value="cerrada">Cerradas</option>
  </select>
  <p id="contador" role="status"></p>
  <ul id="listado"></ul>
</body>
</html>
```

Código para depurar · app.js · parte 1 de 3

```javascript
const form = document.querySelector("#formIncidencia");
const titulo = document.querySelector("#titulo");
const limpiar = document.querySelector("#limpiar");
const filtro = document.querySelector("#filtro");
const contador = document.querySelector("#contador");
const listado = document.querySelector("#listado");

let incidencias = [
  { id: 1, titulo: "Proyector del aula 12", prioridad: 3, abierta: true },
  { id: 2, titulo: "Wifi lento en la biblioteca", prioridad: 5, abierta: true },
  { id: 3, titulo: "Teclado sin la tecla Ñ", prioridad: 1, abierta: false }
];
let siguienteId = 4;

function validar() {
  if (titulo.value.trim().length < 5) {
    titulo.setCustomValidity("Escribe al menos 5 caracteres.");
  }
  return form.reportValidity();
}
```

Código para depurar · app.js · parte 2 de 3 · continuación

```
function guardar(evento) {
  evento.preventDefault();
  if (!validar()) return;
  const datos = new FormData(form);
  incidencias.push({
    id: siguienteId++,
    titulo: String(datos.get("titulo")).trim(),
    prioridad: Number(datos.get("prioridad")),
    abierta: true
  });
  form.reset();
  renderizar();
}

function aplicarFiltro() {
  if (filtro.value === "abierta") {
    incidencias = incidencias.filter((i) => i.abierta);
  } else if (filtro.value === "cerrada") {
    incidencias = incidencias.filter((i) => !i.abierta);
  }
  renderizar();
}

function gestionarClic(evento) {
  const accion = evento.target.dataset.accion;
  const id = evento.target.dataset.id;
  if (accion === "alternar") {
    const incidencia = incidencias.find((i) => i.id === id);
    incidencia.abierta = !incidencia.abierta;
  }
  if (accion === "borrar") {
    incidencias = incidencias.filter((i) => i.id !== id);
  }
  renderizar();
}

function crearBoton(accion, id, texto) {
  const boton = document.createElement("button");
  boton.type = "button";
  boton.dataset.accion = accion;
  boton.dataset.id = id;
  boton.innerHTML = `<span>${texto}</span>`;
  return boton;
}
```

Código para depurar · app.js · parte 3 de 3 · continuación

```
function renderizar() {
  listado.replaceChildren();
  for (const i of incidencias) {
    const fila = document.createElement("li");
    fila.innerHTML = `<strong>${i.titulo}</strong> · P${i.prioridad} `;
    fila.append(
      crearBoton("alternar", i.id, i.abierta ? "Cerrar" : "Reabrir"),
      crearBoton("borrar", i.id, "Borrar")
    );
    listado.append(fila);
  }
  listado.addEventListener("click", (e) => gestionarClic(e));
  contador.textContent = `Total: ${incidencias.length}`;
}

form.addEventListener("submit", guardar);
filtro.addEventListener("change", aplicarFiltro());
limpiar.addEventListener("click", () => form.reset());
renderizar();
```

B Síntomas que debes reproducir

Reproduce cada síntoma, localiza la causa con la consola o un breakpoint, corrígela y anótala. Al arreglar un fallo pueden aparecer otros que antes quedaban ocultos.

N.º	Síntoma observado
1	La consola muestra un error al cargar y la página no reacciona.
2	Cambiar el selector «Ver» no produce ningún efecto.
3	Pulsar «Limpiar» dispara la validación o intenta añadir una incidencia.
4	Tras un título demasiado corto, el formulario sigue inválido aunque lo corrijas.
5	Se admite una incidencia sin prioridad y aparece como `P0`.
6	Al filtrar por «Abiertas» y volver a «Todas», las cerradas han desaparecido.
7	Pulsar el texto de un botón de la lista no hace nada.
8	Aunque aciertes en el botón, «Cerrar» provoca un error y «Borrar» no elimina nada.
9	Tras varios renderizados, un único clic ejecuta la acción más de una vez.
10	El título `<b>Hola</b>` se ve en negrita en lugar de literalmente.

FORMATO DE ENTREGA DE CADA FALLO

N.º	Dónde está	Causa	Corrección	Cómo lo has comprobado
…	Función o línea	Por qué falla	Qué has cambiado	Prueba realizada

C Parte C · Amplía la aplicación corregida (3 puntos)

1. Convierte cada incidencia en una instancia de la clase `Incidencia`, con el método `alternar()` y el getter `urgente`, verdadero con prioridad 4 o superior. **1 p**
2. Muestra en `#contador` el texto `Visibles: X · Abiertas: Y · Urgentes abiertas: Z · Total: T`. Solo «Visibles» depende del filtro. **1 p**
3. Añade un campo de búsqueda que se combine con el filtro. `Escape` lo vacía; mantener la tecla pulsada no repite el trabajo. **0,5 p**
4. Muestra un mensaje cuando no haya resultados y devuelve el foco a `#titulo` después de un alta correcta. **0,5 p**

COMPRUEBA EL RESULTADO FINAL

- Datos iniciales: `Visibles: 3 · Abiertas: 2 · Urgentes abiertas: 1 · Total: 3`.
- Cierra «Wifi lento» y filtra por «Abiertas»: `Visibles: 1 · Abiertas: 1 · Urgentes abiertas: 0 · Total: 3`.
- Añade dos incidencias con el mismo título y borra una: la otra permanece. Cada clic produce una sola operación.
- Pulsa el `span` interior de cada botón y prueba de nuevo el título corto después de corregirlo.

Defensa: ¿Qué fallo se ocultaba detrás de otro? ¿Por qué registrar la escucha dentro de `renderizar` duplica acciones aquí, pero no lo haría con una función con nombre? Elige un fallo y muéstralo con un breakpoint.

EX5 Taquilla de un festival

10 puntos | 100 minutos | Dificultad: máxima

Situación: un festival de dos días (sábado y domingo) necesita una taquilla web. El personal da de alta entradas, las edita, las valida en la puerta y consulta la recaudación. Reúne todo el trabajo de la evaluación.

INTEGRA · TEMAS 1–4 · TEMA 5 · TEMA 6 COMPLETO

HTML Material de partida

HTML · cuerpo de index.html

```html
<h1>Taquilla del festival</h1>
<form id="formEntrada" novalidate>
  <label for="comprador">Nombre</label>
  <input id="comprador" name="comprador" required>
  <label for="correo">Correo</label>
  <input id="correo" name="correo" type="email" required>
  <label for="tipo">Tipo de entrada</label>
  <select id="tipo" name="tipo" required>
    <option value="">Elige un tipo</option>
    <option value="general">General</option>
    <option value="vip">VIP</option>
    <option value="grupo">Grupo</option>
  </select>
  <fieldset>
    <legend>Día</legend>
    <label><input type="radio" name="dia" value="sabado" required> Sábado</label>
    <label><input type="radio" name="dia" value="domingo"> Domingo</label>
    <label><input type="radio" name="dia" value="ambos"> Ambos</label>
  </fieldset>
  <label for="personas">Personas del grupo</label>
  <input id="personas" name="personas" type="number" min="4" max="12" disabled>
  <label><input id="backstage" name="backstage" type="checkbox" disabled> Pase de backstage</label>
  <label for="socio">Código de socio (opcional)</label>
  <input id="socio" name="socio" pattern="FEST-[A-Z]{2}[0-9]{2}">
  <button id="enviar" type="submit">Crear entrada</button>
  <button id="cancelar" type="button" hidden>Cancelar edición</button>
</form>
<p id="mensaje" role="status"></p>
<label for="filtroDia">Día</label>
<select id="filtroDia">
  <option value="todos">Todos</option>
  <option value="sabado">Sábado</option>
  <option value="domingo">Domingo</option>
</select>
<label for="buscar">Buscar comprador</label>
<input id="buscar" type="search">
<label><input id="pendientes" type="checkbox"> Solo sin validar</label>
<p id="panel"></p>
<p id="vacio" hidden>No hay entradas que mostrar.</p>
<ul id="entradas"></ul>
```

€ Reglas del negocio

Tipo	Precio	Personas
`General`	35 €	1
`VIP`	80 €. Backstage: 20 € fijos que no se multiplican por el abono.	1
`Grupo`	30 € por persona. Desde 6 personas, un 10 % menos.	4 a 12

- **Abono de ambos días:** multiplica por 1,8 el precio de las entradas (no el backstage).
- **Código de socio:** 10 % de descuento sobre el importe final.
- **Aforo:** 20 personas por día. Una entrada «Ambos» ocupa plaza el sábado y el domingo.

EX5 Se pide

PARTE A · MODELO CON CLASES (2,5 PUNTOS)

1. Crea `Entrada` y las hijas `General`, `VIP` y `Grupo`, con los datos comunes en la base y solo los propios en cada hija. **0,5 p**
2. `precio()` se escribe una vez en la base y aplica el descuento de socio sobre `importe()`, un método que cada hija sobrescribe. **1 p**
3. Añade un getter `personas` polimórfico y un método `validar()`. Una entrada validada no se puede editar ni eliminar. **0,5 p**
4. Crea el método estático `Entrada.ocupacion(lista, dia, idExcluido)`, que devuelve las personas de ese día. **0,5 p**

PARTE B · FORMULARIO Y VALIDACIÓN (2,5 PUNTOS)

1. Según el tipo, habilita y exige `personas` o habilita `backstage`. Los campos que no aplican quedan deshabilitados y vacíos. **0,5 p**
2. Nombre: al menos 3 caracteres tras `trim`. Personas: entero dentro de los límites. Añade `step` si lo necesitas. **0,5 p**
3. Controla el aforo con `setCustomValidity` sobre el campo que corresponda. El mensaje indica el día completo y cuántas plazas quedan. **1 p**
4. Un error corregido no impide el envío siguiente, y un envío inválido nunca cambia el modelo. **0,5 p**

PARTE C · ALTAS, EDICIÓN Y ACCIONES (2,5 PUNTOS)

1. Una única escucha delegada en `#entradas` atiende «Editar», «Eliminar» y «Validar», localizando la entrada por `id`. **0,5 p**
2. «Editar» carga la entrada en el formulario, cambia el botón a «Guardar cambios» y muestra «Cancelar edición». **0,5 p**
3. Al guardar se conserva el `id` y la posición en el array, aunque cambie el tipo. Esto obliga a crear un objeto de otra clase. **0,5 p**
4. Durante una edición, sus propias plazas no cuentan para el aforo. **0,5 p**
5. `Escape`, con el foco en el formulario, cancela la edición. Eliminar la entrada en edición también la cancela. **0,5 p**

PARTE D · VISTA Y PANEL (2,5 PUNTOS)

1. Los tres filtros se combinan sin alterar el array. Una entrada «Ambos» aparece en sábado y en domingo. **0,5 p**
2. Cada fila muestra comprador, tipo, día, personas, precio y estado. Separa `filtrar`, `calcularPanel` y `renderizar` en funciones distintas. **1 p**
3. `#panel` muestra siempre el total: `Sábado X/20 · Domingo Y/20 · Validadas: V de T · Recaudación: Z €`, con `reduce`. **0,5 p**
4. Cuenta con `instanceof` las entradas de cada tipo y muéstralas en consola con una función propia. **0,25 p**
5. Coloca un breakpoint en la escucha delegada e inspecciona el `target`, la acción y el modelo antes y después. Anota lo observado. **0,25 p**

OK Casos de prueba

Entrada o acción	Resultado esperado
General · sábado · General · ambos	`35.00 €` · `63.00 €`
VIP · domingo · backstage	`100.00 €`
VIP · ambos · backstage · con y sin `FEST-AB12`	`147.60 €` · `164.00 €`
Grupo de 4 · sábado · Grupo de 5 · domingo	`120.00 €` · `150.00 €`
Grupo de 6 · ambos	`291.60 €`
Socio `FEST-ab12` · `FEST-AB123`	Inválidos. Vacío es válido.
Sábado: grupos de 8 y 10, y una General y una VIP	`Sábado 20/20`. Otra General de sábado se rechaza; una de domingo se acepta.
Con el sábado lleno, una General «Ambos»	Se rechaza aunque quede sitio el domingo.
Con el sábado lleno, editar el grupo de 10 y dejarlo igual	Se guarda: sus plazas no cuentan.
Editar una General para que sea Grupo de 4	Mismo `id` y posición; `instanceof Grupo` es verdadero.
Validar una entrada	Editar y Eliminar dejan de estar disponibles; el panel suma una validada.
Filtro Domingo + «Solo sin validar» + buscar	Actúan a la vez, y el panel no cambia.

Defensa: Sigue el recorrido completo de un «Guardar cambios»: evento, validación, lectura, modelo y DOM. ¿Qué tres estados coordinas? ¿Por qué `importe()` se sobrescribe y `precio()` no? ¿Qué parte moverías a otro archivo si la aplicación creciera?

✓ Antes de entregar (piensa como si fuese un examen)

- Recarga la página y repite los casos de prueba desde cero, en orden.
- Prueba un caso vacío, uno límite y uno inválido, no solo el caso que esperas que funcione.
- Comprueba que el array y la interfaz cuentan lo mismo después de cada acción.
- Busca `addEventListener` dentro de funciones que se repiten.
- Usa el teclado: Tab, Enter y Escape donde se pida.
- Deja la consola sin errores y retira los `console.log` temporales que no pida el enunciado.

La idea que conecta los cinco ejercicios

La interacción inicia un evento. El manejador valida los datos, actualiza el modelo y, por último, vuelve a pintar la interfaz. Si puedes seguir ese recorrido en tu código, puedes defender tu solución.